American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Course: Introduction to Data Science

Section: K , Group: H

Title: Smartphone Product Price Scraping and Analysis Using R

Supervised by

KAMRUN NAHER KOLI

Submitted By:-

NAME	ID
AYON ROY	22-48435-3

1.Introduction

The purpose of this project is to analyze smartphone product prices through web scraping, data cleaning, exploratory analysis, and visual analytics. The data has been extracted from the e-commerce website https://www.pickaboo.com/product/smartphone, which provides a comprehensive collection of smartphone listings, including product names, prices, and specifications. This project involves scraping smartphone product information, preprocessing and organizing the collected data, analyzing price patterns across different brands and models, and creating meaningful visualizations to understand market trends. By examining the scraped data, the project aims to uncover insights such as price distribution, brand-wise pricing behavior, and feature-based price variations. Overall, this approach helps gain practical experience in handling real-world e-commerce data, applying data analysis techniques, and understanding how product pricing information from online platforms can be transformed into actionable insights for data-driven decision-making.

1.1Research Objective

The primary objective of this study is to analyze the current smartphone market landscape in Bangladesh by extracting real-time data from a leading e-commerce platform, Pickaboo. Specifically, this research aims to identify which brands dominate the market in terms of product volume and average pricing and investigate how hardware specifications (RAM and Storage) influence smartphone pricing across different brands. Smartphones vary significantly in price based on specifications and brand equity. By scraping real-world data, we can move beyond anecdotal evidence to quantify these relationships, satisfying the requirement for a specific, measurable, and data-driven objective.

2. Data Collection (Web Scraping)

The smartphone data used in this project was collected through web scraping from a publicly available e-commerce website (Pickaboo). This website was chosen because it provides detailed and structured information about smartphone products, including brand name, model, price, and customer rating, which are suitable for market analysis.

The data collection process was carried out using R programming language. An API-based scraping approach was used to fetch data in JSON format, which is more reliable and efficient than traditional HTML scraping. The httr package was used to send HTTP requests to the website's API, while the jsonlite package was used to parse and convert the JSON responses into R data frames.

To extract technical specifications such as RAM and storage, regular expressions were applied to product names using the stringr package. The extracted data was then cleaned and stored in a structured format for further analysis. The dplyr package was used for data manipulation and preprocessing.

```
extract_specs <- function(name) {
 if (is.null(name) || name == "" || is.na(name)) {
  return(list(brand = NA, model = NA, ram = NA, storage = NA))
 }
 spec_match <- str_match(name, "(\\d+GB?)\\s*[/+]\\s*(\\d+(GB|TB))")
 if (!is.na(spec_match[1, 1])) {
  ram <- spec_match[1, 2]
  storage <- spec_match[1, 3]
  # Get the part before RAM/Storage as model
  model_part <- str_trim(substr(name, 1, regexpr(spec_match[1,1], name) - 1))
 } else {
  ram <- NA
  storage <- NA
  model_part <- name
 }
scrape_pickaboo_api <- function() {
 base_api_url <- "https://www.pickaboo.com/rest/V1/categorypageapi/smartphone"
 all_products <- data.frame(
  Brand = character(),  model = character(), Ram = character(),  storage = character(), price = numeric(),
rating = numeric(), stringsAsFactors = FALSE )
 seen_ids <- c()
 headers <- add_headers(
  `User-Agent` = "Mozilla/5.0",
  Accept = "application/json"
 )
```

```
# Loop through pages (adjust max pages as needed)
for (page in 1:10) {
 cat("Scraping Page", page, "...\n")
 response <- GET(
  url = base_api_url,
  headers,
  query = list(
   prodLimit = 20,
   currentPage = page,
   featProdLimit = 6,
   web = 1
  )  )
 data <- fromJSON(content(response, "text", encoding = "UTF-8"))
 product_list <- data$cat_prods
 if (page == 1 && !is.null(data$featured_products)) {
  product_list <- bind_rows(product_list, data$featured_products)
 }
 if (length(product_list) == 0) next
  for (i in 1:nrow(product_list)) {
  p_id <- product_list$id[i]
  if (!is.na(p_id) && !(p_id %in% seen_ids)) {
   seen_ids <- c(seen_ids, p_id)
   name <- product_list$product_name[i]
   specs <- extract_specs(name
   all_products <- rbind(all_products, data.frame(
```

```
      Brand = specs$brand,
      model = specs$model,
      Ram = specs$ram,
      storage = specs$storage,
      price = ifelse(is.null(product_list$product_price[i]), NA, product_list$product_price[i]),
      rating = ifelse(is.null(product_list$rating[i]), NA, product_list$rating[i]),
      stringsAsFactors = FALSE
    ))
   }
  }
  cat("Processed", nrow(product_list), "products on page", page, "\n\n")
  Sys.sleep(1)
 }
 write.csv(all_products, "pickaboo_api_data.csv", row.names = FALSE)
 cat("Saved", nrow(all_products), "products to 'pickaboo_api_data.csv'\n")
 return(all_products)
}
products_df <- scrape_pickaboo_api()
```

2.1Tools and Libraries

- httr: To handle HTTP GET requests and manage API headers.
- jsonlite: To parse the returned JSON content into a structured R data frame.
- stringr: To extract technical specifications from unstructured product titles using Regular Expressions (Regex).
- dplyr: For data manipulation and organization during the scraping process

3. Data Understanding and Exploration

Data Understanding and Exploration means studying the dataset to understand its structure, features, and quality. It involves using basic statistics and visualizations to find patterns, relationships, and problems in the data.The initial dataset included the following variables: Brand, Model, RAM, Storage, Price, and Rating. The data types included character strings for specifications and numerical values for price and ratings.

Importing and Showing The Dataset

head() function shows the first 6 rows

```
> head(products_df)
     Brand   model  Ram storage price rating
1   realme      C85  6GB   128GB 18999     NA
2   realme C85 Pro  8GB   256GB 25999     NA
3   realme C85 Pro  8GB   128GB 24999     NA
4     XTRA      R24 <NA>    <NA>  2499    4.9
5 Symphony  EVO 10 <NA>    <NA>  2250    5.0
6   realme       12  8GB   256GB 24999    5.0
>
```

str(df) : It shows a quick summary of your entire dataset.

```
> str(products_df)
'data.frame':   165 obs. of  6 variables:
 $ Brand  : chr  "realme" "realme" "realme" "XTRA" ...
 $ model  : chr  "C85" "C85 Pro" "C85 Pro" "R24" ...
 $ Ram    : chr  "6GB" "8GB" "8GB" NA ...
 $ storage: chr  "128GB" "256GB" "128GB" NA ...
 $ price  : int  18999 25999 24999 2499 2250 24999 0 0 0 0 ...
 $ rating : num  NA NA NA 4.9 5 5 NA NA NA NA ...
>
```

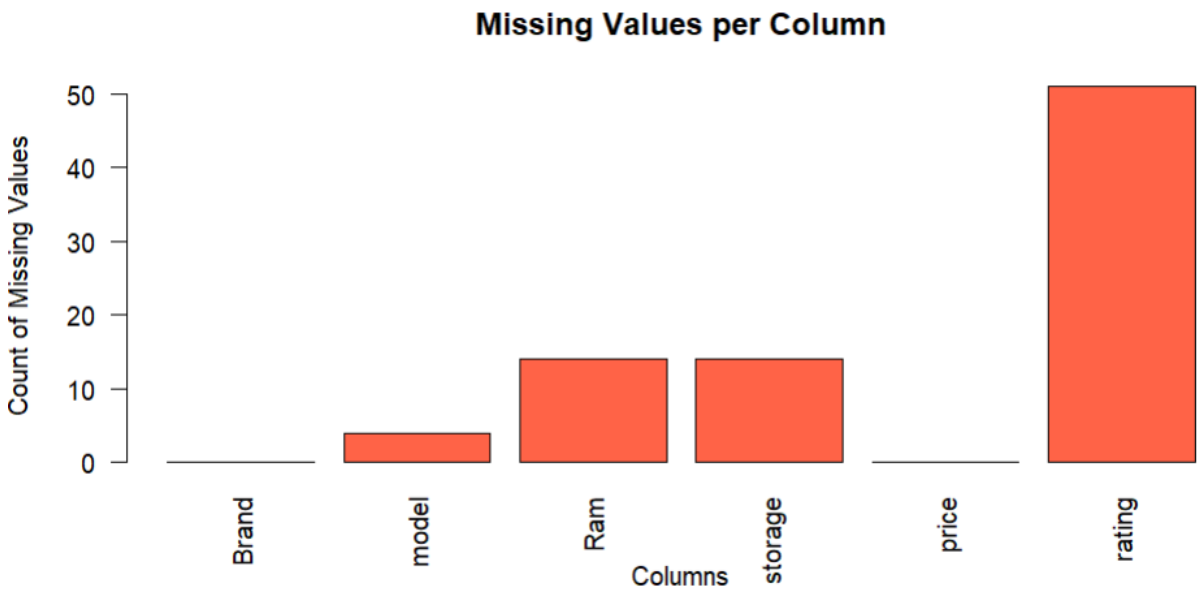
Check Missing Values

colSums(is.na(products_df))

```
colSums(is.na(products_df))
Brand   model     Ram storage   price  rating
    0       4      14      14       0      51
```

```
missing_counts <- colSums(is.na(products_df))

barplot(missing_counts,

    main = "Missing Values per Column",

    xlab = "Columns",

    ylab = "Count of Missing Values",

    col = "tomato",

    las = 2)
```

A bar chart was generated to visualize missing values per column. We identified that RAM and Storage often contained NA values for feature phones ("Button Phones").

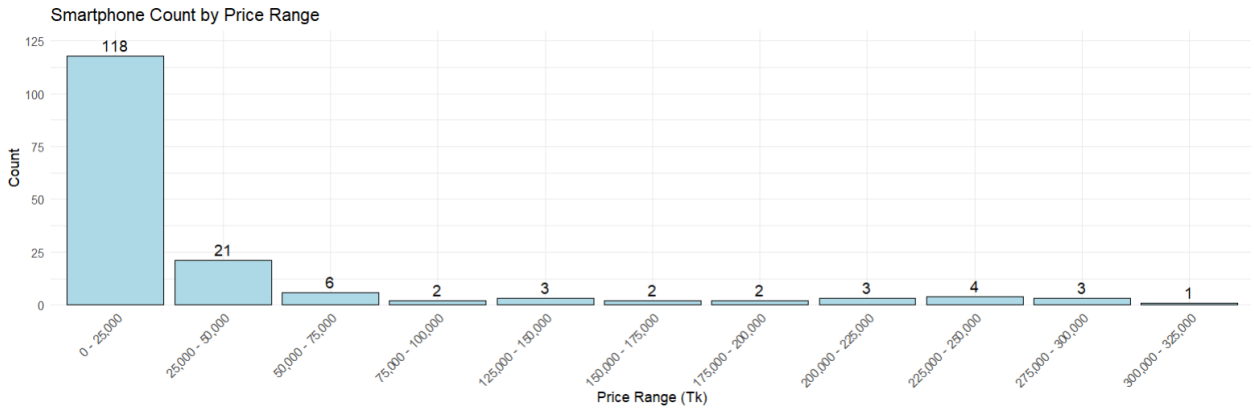


Price Distribution:

```
price_summary <- products_df %>%

 filter(!is.na(price)) %>%

 mutate(price_range = cut(price,

                breaks = seq(0, 350000, by = 25000),

                include.lowest = TRUE,

                right = FALSE,
```

```
    labels = paste0(seq(0, 325000, by = 25000), "-", seq(25000, 350000, by = 25000))
  )) %>%
  group_by(price_range) %>%
  summarise(Count = n(), .groups = "drop")
ggplot(price_summary, aes(x = price_range, y = Count)) +
  geom_bar(stat = "identity", fill = "lightblue", color = "black") +
  geom_text(aes(label = Count), vjust = -0.4, size = 4) +
  scale_y_continuous(breaks = seq(0, max(price_summary$Count) + 10, by = 5)) +
  labs(title = "Smartphone Count by Price Range",
       x = "Price Range (Tk)", y = "Count") +
  theme_minimal()
```

A histogram with custom bins (25,000 Tk intervals) revealed the distribution of smartphone prices, highlighting whether the market is skewed toward budget or flagship devices.

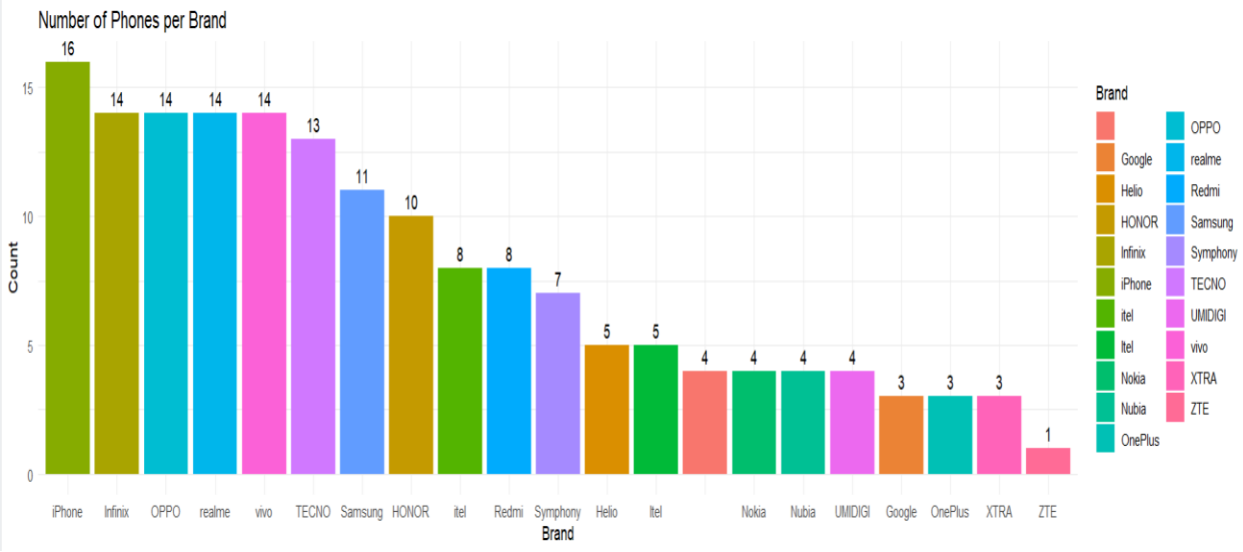


Count of phones per brand

```
products_df %>%
  group_by(Brand) %>%
  summarise(Count = n()) %>%
  ggplot(aes(x = reorder(Brand, -Count), y = Count, fill = Brand)) +
  geom_bar(stat = "identity") +
```

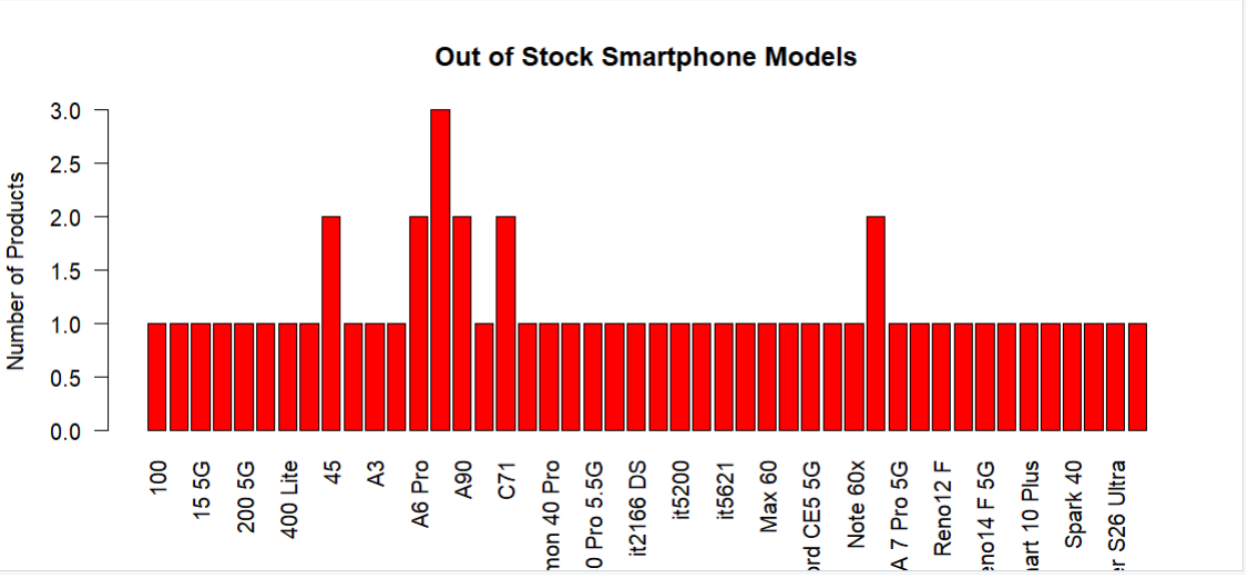
```
 geom_text(aes(label = Count), vjust = -0.5) +
 labs(title = "Number of Phones per Brand", x = "Brand", y = "Count") +
 theme_minimal()
```

A bar chart visualized the number of distinct devices per brand, identifying market leaders in terms of inventory size.



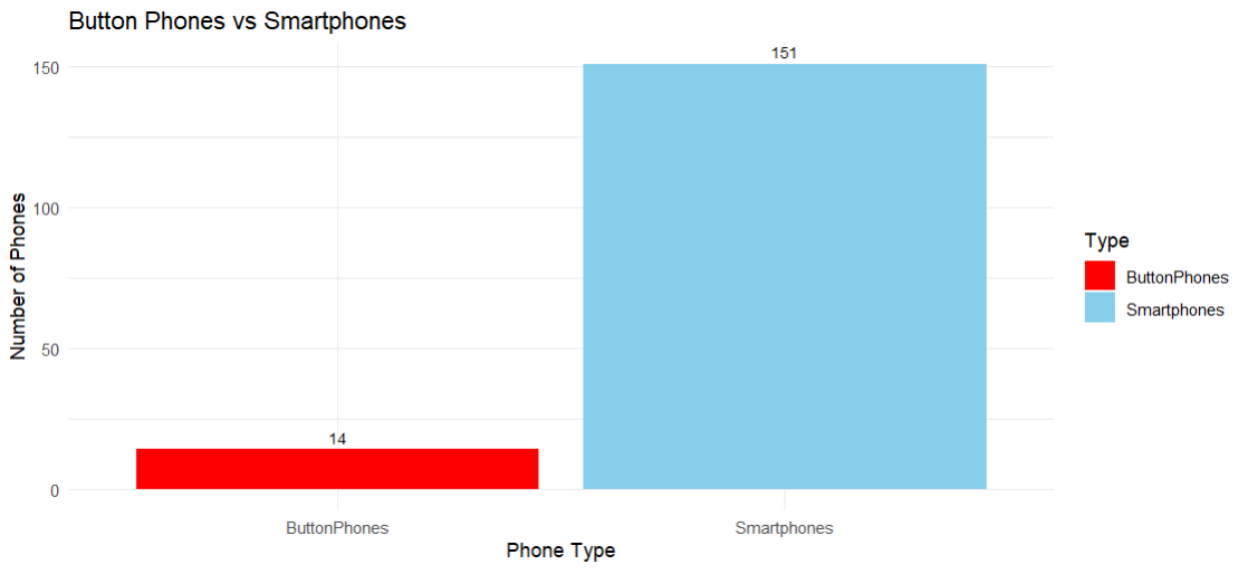
which model phone out of stock

```
out_of_stock <- products_df %>%
 filter(price == 0)
nrow(out_of_stock)
barplot(
 table(out_of_stock$model),
 las = 2,
 col = "red",
 main = "Out of Stock Smartphone Models",
 ylab = "Number of Products"
)
```



Number of button and smartphones

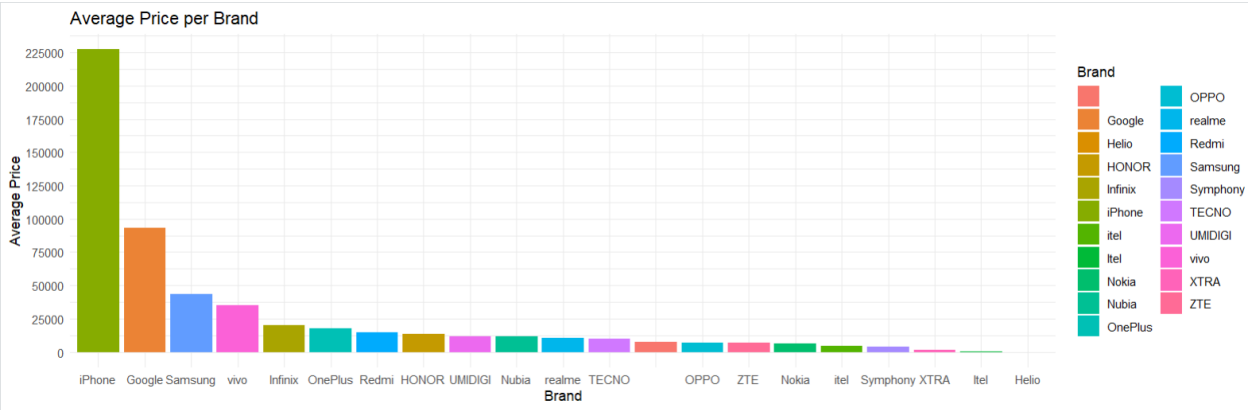
```
summary_df <- products_df %>%
  summarise(
    ButtonPhones = sum(is.na(Ram) | is.na(storage)),
    Smartphones = sum(!is.na(Ram) & !is.na(storage))
  ) %>%
  pivot_longer(everything(), names_to = "Type", values_to = "Count")
ggplot(summary_df, aes(x = Type, y = Count, fill = Type)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = Count), vjust = -0.5, size = 3) +
  scale_fill_manual(values = c("red", "skyblue")) +
  labs(title = "Button Phones vs Smartphones",
       x = "Phone Type",
       y = "Number of Phones") +
  theme_minimal()
```



Average price per brand

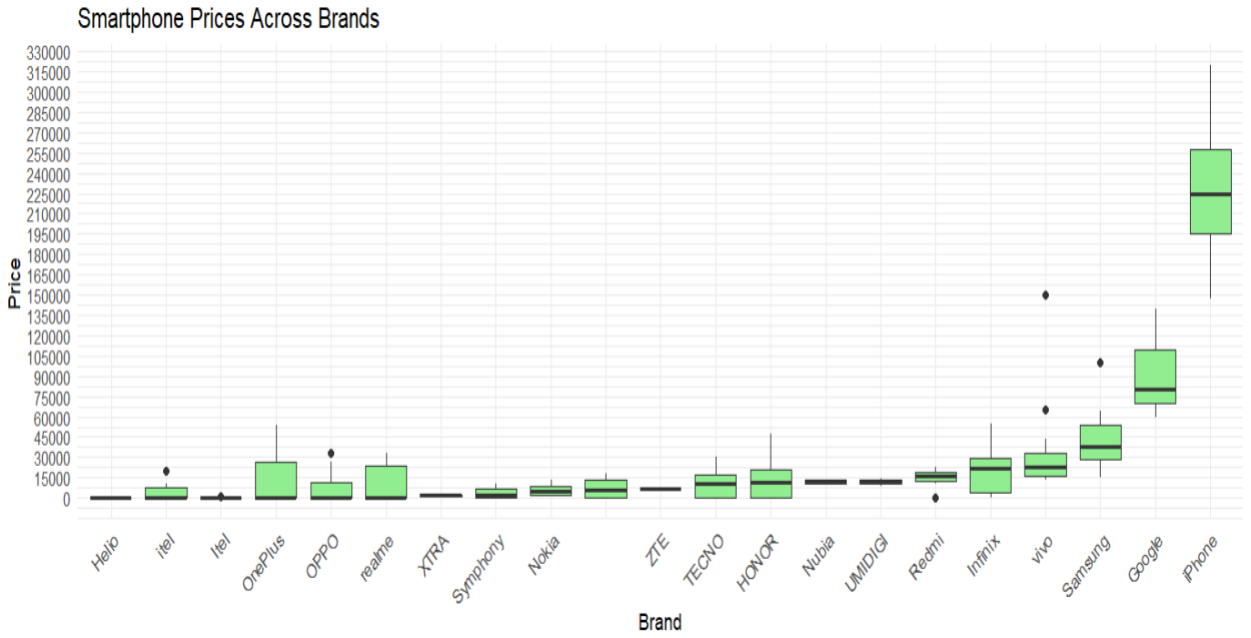
```
products_df %>%
 group_by(Brand) %>%
 summarise(AveragePrice = mean(price, na.rm = TRUE), .groups = "drop") %>%
 ggplot(aes(x = reorder(Brand, -AveragePrice), y = AveragePrice, fill = Brand)) +
 geom_bar(stat = "identity") +
 scale_y_continuous(breaks = seq(0, 350000, by = 25000)) +
 labs(title = "Average Price per Brand", x = "Brand", y = "Average Price") +
 theme_minimal()
```

The *Average Price per Brand* analysis shows how smartphone prices vary across different manufacturers by calculating the mean price of all available models for each brand.



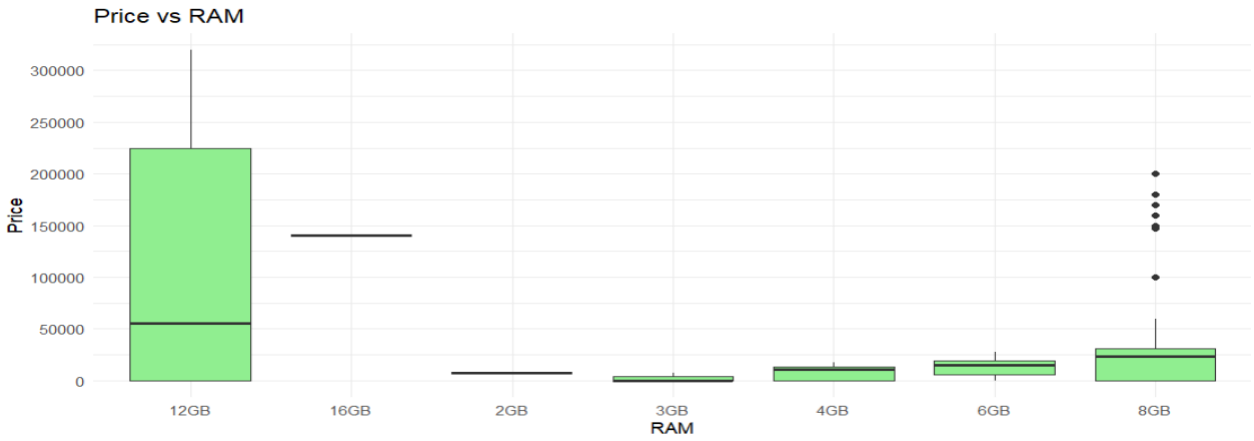
Price distribution by Brand

```
ggplot(products_df %>% filter(!is.na(price), !is.na(Brand)),
    aes(x = reorder(Brand, price, FUN = median), y = price)) +
 geom_boxplot(fill = "lightgreen") +
 labs(title = "Smartphone Prices Across Brands",
    x = "Brand", y = "Price ") +
 scale_y_continuous(breaks = seq(0, 350000, by = 15000)) +
 theme_minimal() +
 theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



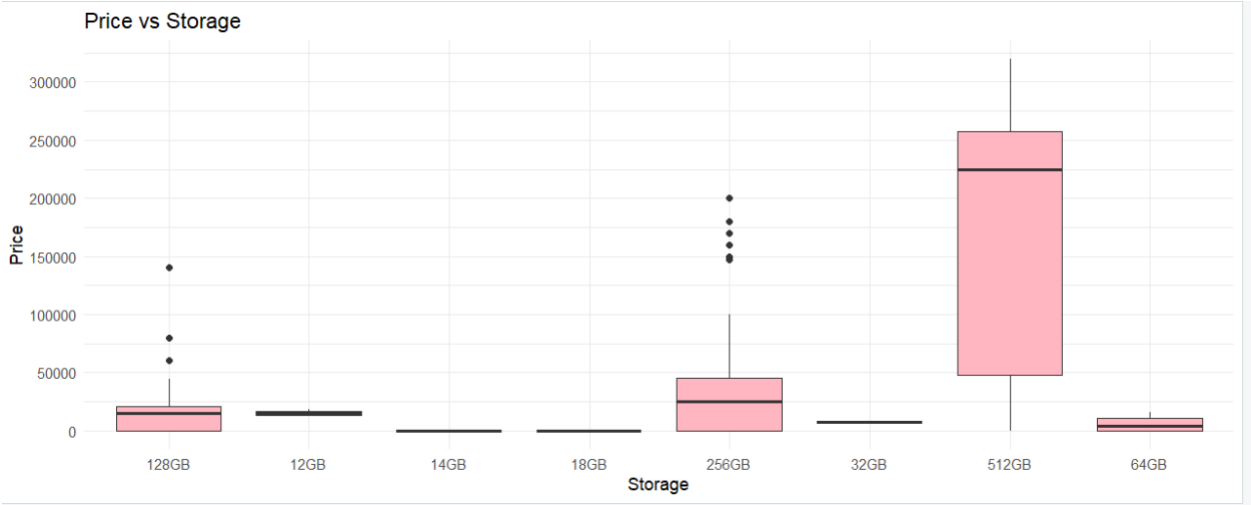
price vs Ram

```
ggplot(products_df %>% filter(!is.na(Ram)), aes(x = Ram, y = price)) +
 geom_boxplot(fill = "lightgreen") +
 labs(title = "Price vs RAM", x = "RAM", y = "Price") +
 scale_y_continuous(breaks = seq(0, 350000, by = 50000)) +
 theme_minimal()
```



price vs storage

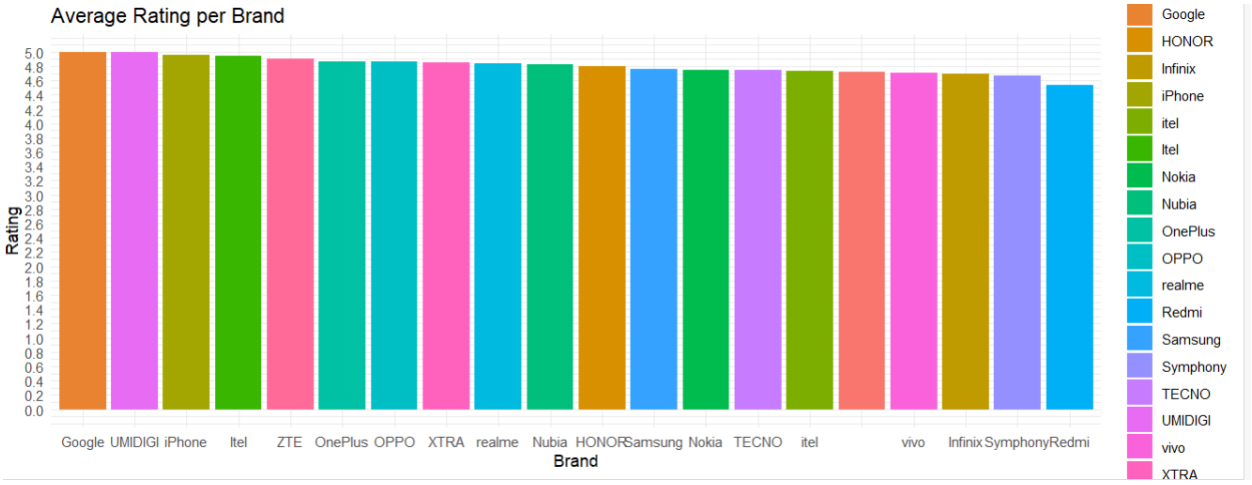
```
ggplot(products_df %>% filter(!is.na(storage)), aes(x = storage, y = price)) +
 geom_boxplot(fill = "lightpink") +
 labs(title = "Price vs Storage", x = "Storage", y = "Price") +
 scale_y_continuous(breaks = seq(0, 350000, by = 50000)) +
 theme_minimal()
```



Rating Analysis with brand

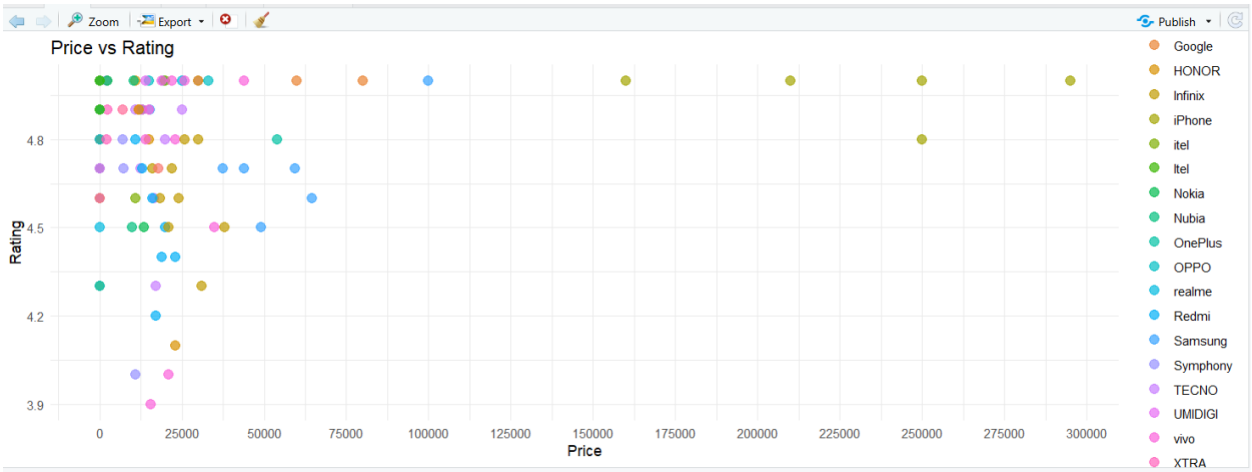
```
products_df %>%
 filter(!is.na(rating)) %>%
 group_by(Brand) %>%
 summarise(AverageRating = mean(rating)) %>%
```

```
ggplot(aes(x = reorder(Brand, -AverageRating), y = AverageRating, fill = Brand)) +
geom_bar(stat = "identity") +
scale_y_continuous(breaks = seq(0,5, by = .2)) +
labs(title = "Average Rating per Brand", x = "Brand", y = "Rating") +
theme_minimal()
```



Price vs Rating

```
ggplot(products_df %>% filter(!is.na(rating)), aes(x = price, y = rating, color = Brand)) +
 geom_point(size = 3, alpha = 0.7) +
 scale_x_continuous(breaks = seq(0, 350000, by = 25000)) +
 labs(title = "Price vs Rating", x = "Price", y = "Rating") +
 theme_minimal()
```



chi -square test

```
products_df <- products_df %>%
 mutate(
   Brand = as.factor(Brand),
   Sold_out = as.factor(ifelse(price == 0, "Yes", "No"))   )
tbl <- table(products_df$Brand, products_df$Sold_out)
chi_result <- chisq.test(tbl)
print(chi_result)
```

The p-value is very small, so Brand and Sold_out are related.Some brands have more Sold Out phones than others.

```
        Pearson's Chi-squared test

data:  tbl
X-squared = 64.329, df = 20, p-value = 0.000001492
```

4. Data Preprocessing

To prepare the dataset for analysis and modeling, the following preprocessing steps were applied:

Handling Missing model

```
products_df$model[is.na(products_df$model)] <- "Unknown"
colSums(is.na(products_df))
```

Missing model names were replaced with "Unknown".

```
> products_df$model[is.na(products_df$model)] <- "Unknown"
> colSums(is.na(products_df))
  Brand   model     Ram storage   price  rating
      0       0      14      14       0      51
```

Handle missing rating using mean

```
rating_mean <- mean(products_df$rating, na.rm = TRUE)
products_df$rating[is.na(products_df$rating)] <- rating_mean
```

products_df$rating <- round(products_df$rating, 1)

head(products_df)

Missing ratings were filled using the mean rating.

```
     Brand   model  Ram storage price rating
1   realme      C85  6GB   128GB 18999    4.8
2   realme C85 Pro  8GB   256GB 25999    4.8
3   realme C85 Pro  8GB   128GB 24999    4.8
4     XTRA      R24 <NA>    <NA>  2499    4.9
5 Symphony  EVO 10 <NA>    <NA>  2250    5.0
6   realme       12  8GB   256GB 24999    5.0
> |
```

Feature Engineering

- RAM and Storage values were extracted from product names using regular expressions.
- RAM and Storage were converted into numeric values

Outlier Detection

```
df <- products_df %>%
 mutate(price = ifelse(price == 0, NA, price)) %>%
 filter(!is.na(price), !is.na(Brand))
Q1 <- quantile(df$price, 0.25)
Q3 <- quantile(df$price, 0.75)
IQR_value <- Q3 - Q1
lower <- Q1 - 1.5 * IQR_value
upper <- Q3 + 1.5 * IQR_value
df_outliers <- df %>%
 filter(price < lower | price > upper)
outlier_summary <- df_outliers %>%
 group_by(Brand) %>%
 summarise(max_price = max(price), .groups = "drop")
```

print(outlier_summary)

The IQR method was used to detect price outliers. Outliers were analyzed brand-wise.

	Brand	max_price
	<chr>	<int>
1	Google	139999
2	Samsung	99999
3	iPhone	319999

Scaling

Numerical features Price, RAM, Storage were standardized using scale() before clustering.

cluster_scaled <- scale(cluster_df)

5. Modeling and Analysis

Choice of Model

K-Means Clustering was selected because:

It is good for finding groups in data when we don't have labels .It puts similar items together automatically. It is often used in market segmentation to find customer or product groups.

Model Training

The Method was used to determine the optimal number of clusters.Based on the elbow curve, K = 3 clusters were selected.K-Means was applied on scaled features: Price, RAM, and Storage.

Cluster Interpretation

Each smartphone was assigned to one of three clusters.

```
cluster_range <- cluster_df %>%
 group_by(KMeans_Cluster) %>%
 summarise(
  Price_Range =
   paste0(round(min(price)), " to ", round(max(price))),
  RAM_Range =
   paste0(round(min(Ram_num)), " to ", round(max(Ram_num))),
  Storage_Range =
```

```
    paste0(round(min(Storage_num)), " to ", round(max(Storage_num))))

print(cluster_range)
```

	KMeans_Cluster	Price_Range	RAM_Range	Storage_Range
	<fct>	<chr>	<chr>	<chr>
1	1	0 to 199999	8 to 16	14 to 512
2	2	0 to 43999	2 to 8	12 to 256
3	3	209999 to 319999	12 to 12	512 to 512

```
>
```

6. Results and Interpretation

Market Composition

The Button Phones vs Smartphones chart shows how many low-tech vs high-tech phones are in the dataset. Most phones are smartphones, but some feature phones are still present.

Price Analysis

Price vs Brand: Apple and Samsung phones are more expensive and have a wider price range than budget brands like Symphony or Walton.

Price vs RAM/Storage: Phones with more RAM or storage usually cost more.

Rating Trends

Average Rating per Brand shows customer satisfaction. Price and rating don't always match. some cheaper phones have good ratings, and some expensive phones don't have the highest ratings.

Cluster Summary

Budget cluster: cheap phones with low RAM and storage

Mid-range cluster: moderately priced phones with decent RAM and storage

Premium cluster: very expensive phones with high RAM and storage

7. Conclusion and Future Work

Conclusion

This project successfully demonstrated a complete data science pipeline from web scraping to clustering analysis. Using real-world smartphone data, meaningful price-based market segments were identified.

Limitations

- Data collected from a single e-commerce website.
- Limited features such as camera, battery, or processor were not included.

Future Work

- Add additional product details like battery capacity and camera quality to make the analysis more complete.
- Use techniques like DBSCAN or Hierarchical Clustering to find better or more natural market segments.
- Build machine learning models to estimate smartphone prices based on features like RAM, storage, brand, battery, and camera